RAPPORT DE TEST D'INTRUSION Application web SecureShop

Audit de sécurité applicative en environnement local : détection, exploitation et correction des vulnérabilités



Auteur : Bouhadfane ElBatoul
applicative Date : 20 juin 2026
Code source : github.com/bouhadfane-elbatoul/securesho

1.Introduction

La sécurité des applications web est un enjeu majeur, car une seule faille peut exposer les données des utilisateurs ou causer des pertes financières. Ce rapport présente l'audit de SecureShop, une application e-commerce volontairement vulnérable développée avec React et FastAPI, exécutée uniquement dans un environnement local et contrôlé.

L'objectif est de détecter, exploiter puis corriger sept vulnérabilités : injection SQL, IDOR, contrôle d'accès cassé, XSS stockée, exposition excessive de données, upload de fichier non restreint et manipulation de prix.

La démarche suit une méthodologie de test d'intrusion : reconnaissance, tests d'authentification, d'autorisation et de validation des entrées, puis tests avancés. Pour chaque faille, le rapport présente sa définition, la menace qu'elle représente pour le site, une preuve de concept avant correction, le correctif appliqué et son re-test.

2. Synthèse des résultats

#	**Vulnérabilité**	**Endpoint**	**CWE**	**Sévérité**
1	Injection SQL	GET /api/products/search	CWE-89	Critique
2	IDOR	GET /api/orders/{id}	CWE-639	Élevée
3	Contrôle d'accès cassé	GET /api/admin/users	CWE-862	Élevée
4	XSS stockée	Avis produits	CWE-79	Élevée
5	Exposition excessive de données	GET /api/users/{id}	CWE-200	Moyenne à élevée
6	Upload de fichier non restreint	POST /api/users/me/avatar	CWE-434	Élevée
7	Manipulation de prix (logique métier)	POST /api/checkout	CWE-840	Élevée

3. Détail des vulnérabilités

3.1 Injection SQL:

Définition. Une injection SQL survient quand une entrée utilisateur est insérée directement dans une requête SQL. L'attaquant peut alors modifier la logique de la requête.

Menace sur SecureShop. La barre de recherche permettait de lire des données non prévues, voire d'atteindre d'autres tables comme `users`.

Exemple avant correction. Recherche normale : `keyboard` renvoie 1 produit. Recherche `' OR '1'='1` : les 8 produits sont renvoyés, donc l'entrée a modifié la requête. Cause : `f"... LIKE '%{q}%'"`, la concaténation.

Correction. Remplacement par l'ORM SQLAlchemy : `db.query(Product).filter(or_(Product.name.ilike(f"%{q}%"), Product.description.ilike(f"%{q}%")))`. L'entrée est désormais une donnée, jamais du code.

Re-test. La même charge ne renvoie plus rien.

3.2 IDOR (Insecure Direct Object Reference)

Définition. Le serveur renvoie une ressource à partir d'un identifiant fourni par le client sans vérifier que l'utilisateur en est propriétaire.

Menace sur SecureShop. Un client connecté pouvait lire les commandes (adresse, articles, total) des autres clients.

Exemple avant correction. Connectée en tant qu'alice (id 1), la requête `GET /api/orders/2` renvoyait la commande de bob. Il suffisait de changer le numéro dans l'URL.

Correction. Ajout de `if order.user_id != current_user.id and current_user.role != "admin": raise HTTPException(403)`.

Re-test. `/api/orders/2` renvoie 403 pour alice, `/api/orders/1` fonctionne toujours.

3.3 Contrôle d'accès cassé (fonction admin)

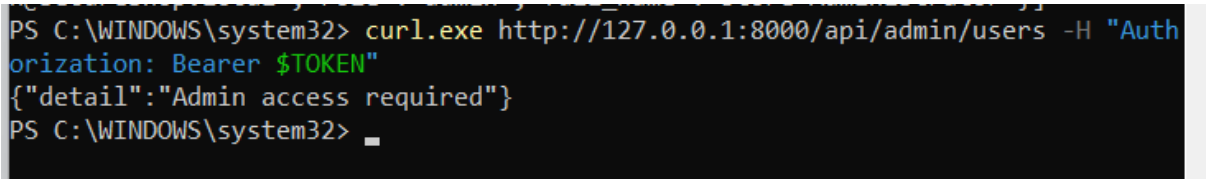
Définition. Un endpoint réservé aux administrateurs ne vérifie pas le rôle de l'appelant (authentification sans autorisation).

Menace sur SecureShop. Un simple client pouvait obtenir la liste de tous les comptes (usernames, emails, rôles), ce qui facilite d'autres attaques ciblées.

Exemple avant correction. Avec le token d'alice : `/api/admin/stats` répondait 403 (protégé), mais `/api/admin/users` répondait 200 avec tous les utilisateurs. Un seul endpoint avait été oublié.



Correction. Ajout de `if current_user.role != "admin": raise HTTPException(403, "Admin access required")`.



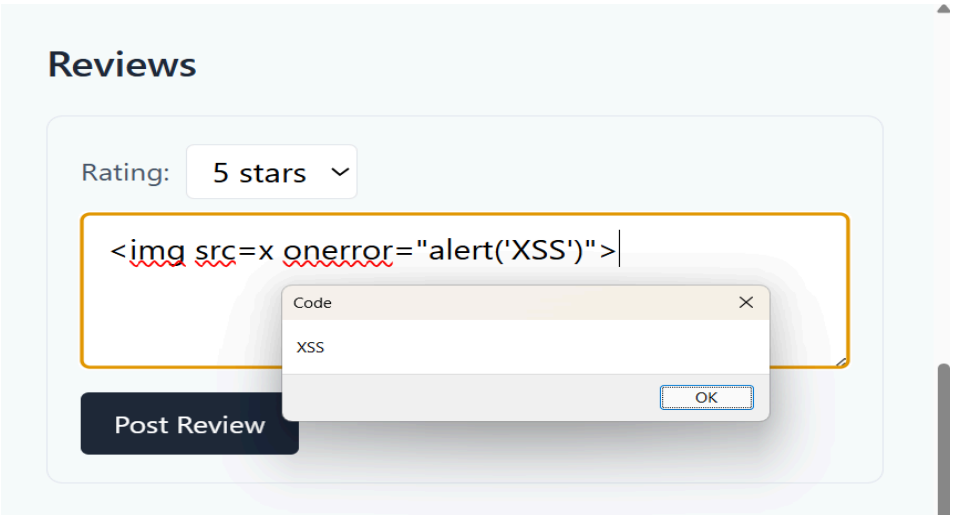
Re-test. alice reçoit 403, admin voit la liste.

3.4 XSS stockée

Définition. Du code JavaScript injecté par un utilisateur est enregistré sur le serveur puis exécuté dans le navigateur des autres visiteurs.

Menace sur SecureShop. Un avis piégé s'exécutait chez tous les visiteurs de la page produit. Il pouvait voler le token JWT stocké dans le `localStorage` et donc usurper des comptes.

Exemple avant correction. Avis contenant `<img src=x onerror="alert('XSS')">` : la popup s'affichait à chaque chargement de la page. Cause : `dangerouslySetInnerHTML={{ __html: r.comment }}`.



Correction. Affichage en texte (`{r.comment}`, React échappe automatiquement) et `html.escape()` côté serveur avant stockage (défense en profondeur).

Re-test. La même charge s'affiche comme texte brut, sans popup.

3.5 Exposition excessive de données

Définition. L'API renvoie plus de champs que nécessaire, par exemple des données sensibles que l'interface n'affiche pas.

Menace sur SecureShop. N'importe qui, sans compte, pouvait récupérer le hash du mot de passe, l'email et l'adresse de chaque utilisateur en parcourant les identifiants 1, 2, 3…

```
PS C:\WINDOWS\system32> curl.exe http://127.0.0.1:8000/api/users/1
{"id":1,"username":"alice","email":"alice@example.com","full_name":"Alice Marti
n","address":"12 Rue des Fleurs, Béjaïa","role":"customer","password_hash":"$2b
$12$Wtzq5t7d2Wd6.LTnO2d6welqkYi9Qj0KC6J9I5zIvsrqCa4wNPcLy","created_at":"2026-0
9-30T14:44:17.449575"}
PS C:\WINDOWS\system32> curl.exe http://127.0.0.1:8000/api/users/2
{"id":2,"username":"bob","email":"bob@example.com","full_name":"Bob Nassim","ad
dress":"45 Avenue de l'Independence, Annaba","role":"customer","password_hash":
"$2b$12$FIO2.srPFdyMfw414tAC0OFhikBwXVS/AcrRRdRtq5o2bJctpy7/C","created_at":"20
26-09-30T14:44:17.449589"}
PS C:\WINDOWS\system32>
```

Exemple avant correction. `GET /api/users/1` sans token renvoyait `password_hash: "2b12$..."`. Le hash permet un cracking hors ligne, sans limite de tentatives.

Correction. Schéma Pydantic `UserPublicOut` (id, username, full_name) via `response_model`, et authentification obligatoire sur l'endpoint.

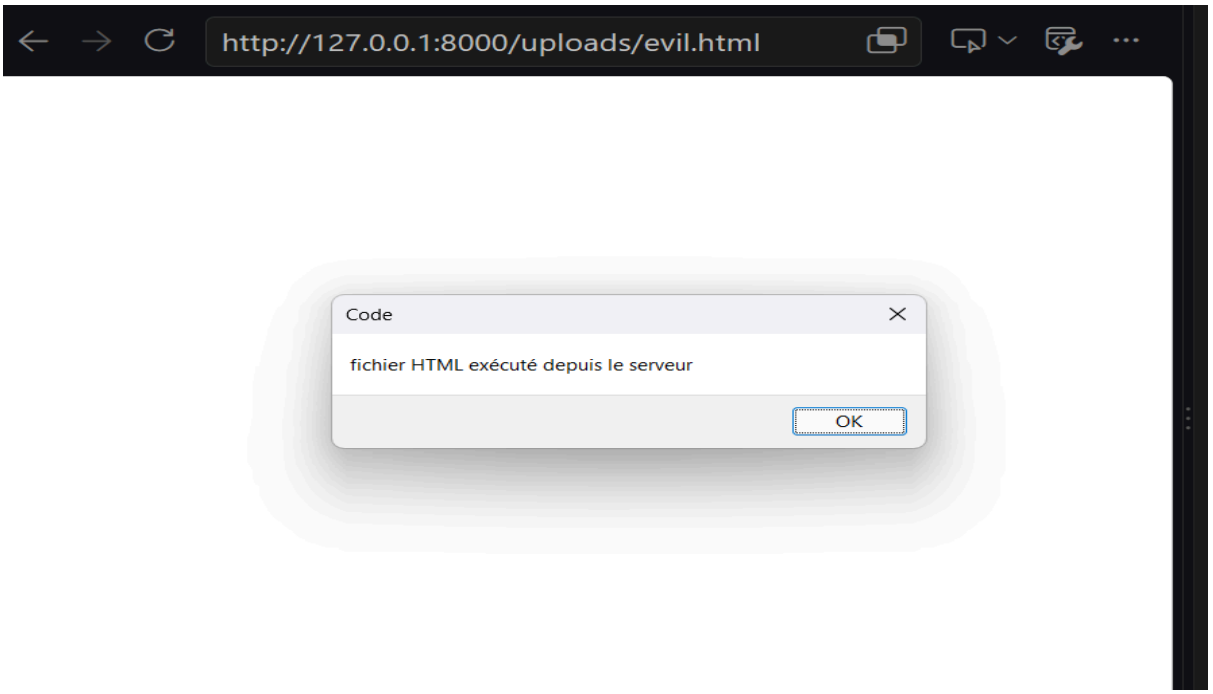
Re-test. Seuls trois champs sont renvoyés, et 401 sans token.

3.6 Upload de fichier non restreint

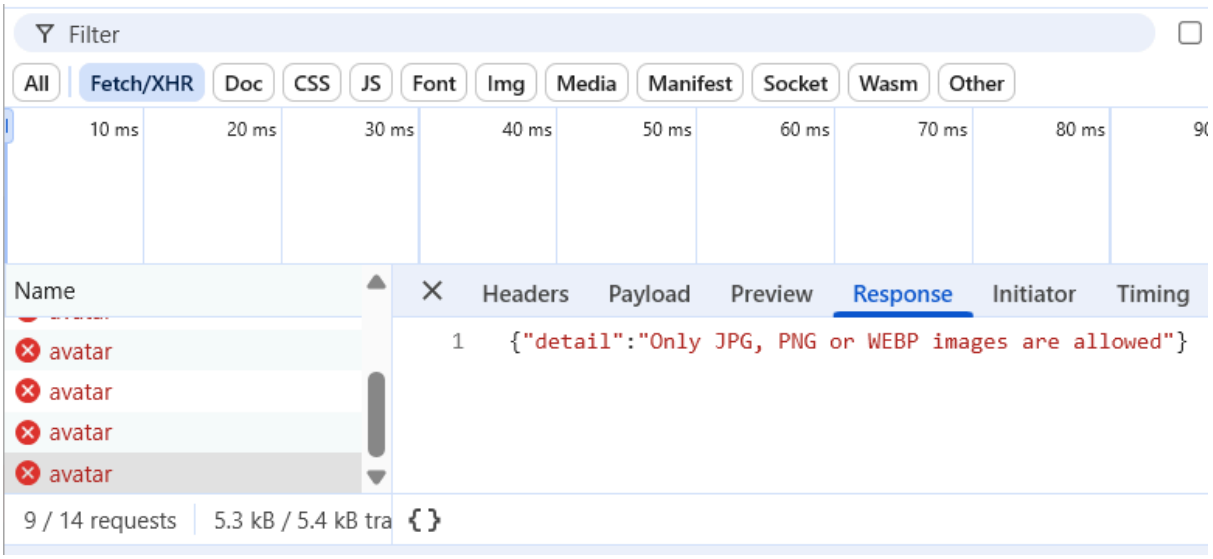
Définition. L'application accepte des fichiers sans valider leur type, leur contenu ni leur nom, puis les stocke dans un dossier accessible.

Menace sur SecureShop. Un attaquant pouvait héberger une page HTML malveillante sur le domaine du site (phishing, XSS), ou tenter d'écraser des fichiers via un nom de fichier piégé.

Exemple avant correction. Upload de `evil.html` contenant un script comme « photo de profil » : accepté, puis `/uploads/evil.html` exécutait le script dans le navigateur.



Correction. Liste blanche d'extensions et de content-types (jpg, png, webp), limite de 2 Mo, nom de fichier généré par le serveur (UUID).



Re-test. `evil.html` est refusé avec `400 Only JPG, PNG or WEBP images are allowed`, une vraie image fonctionne.

3.7 Manipulation de prix (faille de logique métier)

Définition. Le serveur fait confiance à des valeurs métier envoyées par le client (prix, total) au lieu de les recalculer lui-même.

Menace sur SecureShop. N'importe quel client pouvait acheter un produit au prix qu'il choisissait : perte financière directe pour la boutique.

Exemple avant correction. Requête `POST /api/checkout` avec la chaise de bureau (219,00 $) et `unit_price: 0.01` : commande acceptée avec `total: 0.01`. Le frontend calcule le bon prix, mais l'API peut être appelée directement.

```
PS C:\Users\Hp> curl.exe -X POST http://127.0.0.1:8000/api/checkout -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{\"items\":[{\"product_id\":4,\"quantity\":1,\"unit_price\":0.01}],\"total\":0.01,\"shipping_address\":\"test\"}'
{"order_id":4,"total":0.01}
PS C:\Users\Hp>
```

Correction. Le serveur relit le prix en base pour chaque produit, recalcule le total et vérifie la quantité et le stock. Le `unit_price` envoyé est ignoré.

```
PS C:\Users\Hp> curl.exe -X POST http://127.0.0.1:8000/api/checkout -H "Au
thorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{\"ite
ms\":[{\"product_id\":4,\"quantity\":1,\"unit_price\":0.01}],\"total\":0.0
1,\"shipping_address\":\"test\"}'
{"order_id":7,"total":219.0}
PS C:\Users\Hp>
```

Re-test. La même requête renvoie `total: 219.0`.

4. Conclusion

Sept vulnérabilités couvrant plusieurs catégories (injection, contrôle d'accès, XSS, fichiers, logique métier) ont été détectées, exploitées en local, corrigées et validées par un re-test. Elles ont une cause commune : **faire confiance aux données du client**. Les bonnes pratiques retenues :

- Valider et traiter toute entrée comme une donnée, jamais comme du code (requêtes paramétrées, échappement).
- Vérifier l'**autorisation** (propriétaire et rôle) sur chaque endpoint, pas seulement l'authentification.
- Ne renvoyer que les champs nécessaires (`response_model`).
- Valider strictement les fichiers et générer les noms côté serveur.
- Recalculer côté serveur toute valeur sensible (prix, totaux, rôles).
- Appliquer la défense en profondeur (plusieurs couches de protection).

Ce travail montre l'intérêt de tester une application du point de vue de l'attaquant avant sa mise en production.